



WhereAmI

Multiplayer geo-guessing Android application

Course: SY43

Created by:

Léonard GOETZ

Navjit KAMAL JIT

Pierre ZUNDEL

Sacha MOEBEL

Contents

1 Project specification

1.1 Description of the chosen application theme

Our mobile application **WhereAmI** is a real-time multiplayer geo-guessing game designed for friends. While popular games like *GeoGuessr* use street view images from public APIs, **WhereAmI** offers a personalized experience: users challenge their friends by uploading photos they took themselves.

In a typical gameplay scenario, a user uploads a photo from their device. The application tags the image with the user's exact geographical coordinates using the device's built-in GPS sensor. Other players in the game lobby are then tasked with finding where the photo was taken. They guess the location by placing a pin on a map. Points are computed based on the distance between the guess and the actual coordinate.

1.2 Target users and expected use cases

The primary target users are mobile gamers, social groups, outdoor explorers and even relatives living in different regions. Two major use cases describe the user interactions:

1. **Friend group interaction:** A group of friends traveling to different locations upload photos of landmarks or hidden spots. They challenge each other to see who can guess the precise location.
2. **Real-time lobbies :** Competitive players create group rooms and launch custom games. The application updates round statuses, images, and score sheets in real-time, allowing users to play synchronously.

1.3 Functional and non-functional requirements

1.3.1 Functional requirements

- **Authentication & account management:** Users can sign up, log in, manage their profile details (first name, last name, username, profile picture), and reset their password using secure deep-linking ('whereami://reset-password').
- **Social subsystem:** Users can search for other players, send/accept/decline friend requests and form groups.
- **Game creation:** Lobbies can configure games with a custom number of rounds and specific round durations (e.g., 2 hours).
- **Geotagged photo upload:** A player uploads a photo for a game round. The system uses the device's location sensor to capture the GPS coordinates (Latitude/Longitude) at the time of publication.
- **Interactive map guessing:** Opponents view the uploaded image and place a pin on an open-source interactive map (OpenStreetMap) to register their guess.
- **Score calculation:** Scores are automatically calculated using an exponential decay function based on the physical distance between the target and the guessed coordinates.
- **Real-time leaderboard:** The system tracks individual round scores and cumulative game scores, presenting them on a podium layout.

1.3.2 Non-functional requirements

- **Real-time syncing:** Game states, picture uploads, and guess statuses must sync instantly across devices.
- **Security:** Database access must be restricted using Row Level Security (RLS) policies so that users can only view games and uploads belonging to their groups.
- **Usability:** Intuitive Compose-based layouts with clear navigation (Bottom Navigation Bar) and handling of location permissions.

2 Conceptualization of the application

2.1 System architecture and main components

The client application is built using Android architecture guidelines, combining **Clean Architecture** principles with the **MVVM (Model-View-ViewModel)** presentation pattern. It uses **Jetpack Compose** for UI rendering and leverages a serverless cloud infrastructure powered by **Supabase**.

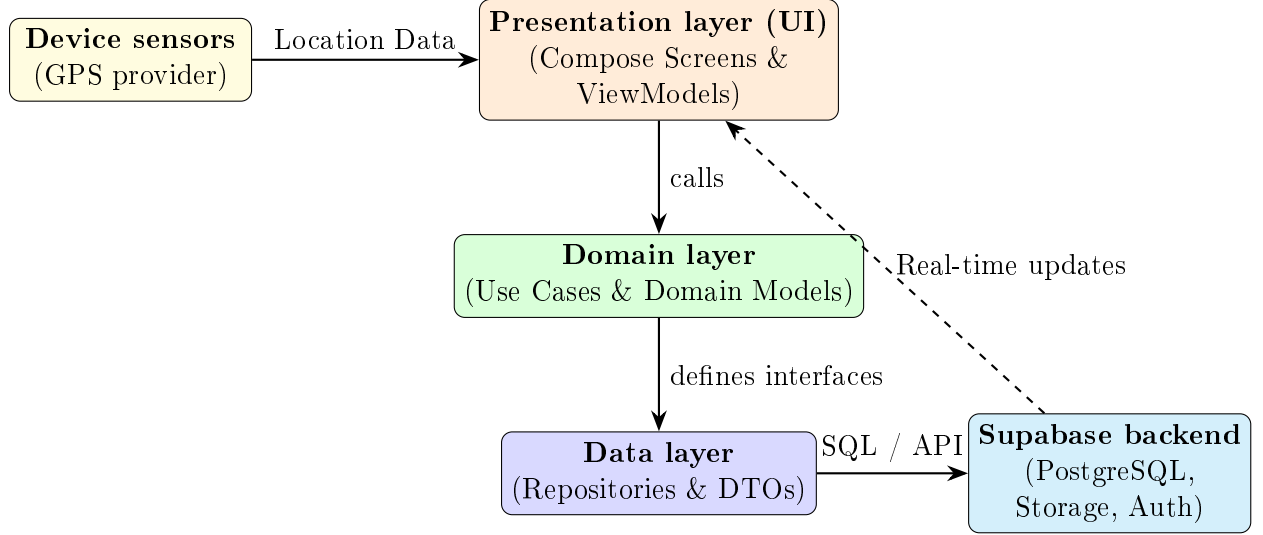


Figure 1: Architecture and data flow diagram of the WhereAmI application.

The codebase is logically separated into three layers:

1. **Presentation layer (UI):** Jetpack Compose functions render state-driven UI screens. The `ViewModel` classes (e.g., `RoundViewModel`) handle presentation logic, consume use cases, and expose state via Kotlin flows.
2. **Domain layer:** Contains business logic. It defines the entities (`Game`, `User`, `Round`, `Guess`) and triggers operations via Use Cases (e.g., `SubmitGuessUseCase`). It defines repository interfaces to maintain independence from specific database technologies.
3. **Data layer:** Implements domain repository interfaces (e.g., `SupabaseGameRepository`). It manages remote communication using Supabase clients, parsing PostgreSQL JSON payloads into Data Transfer Objects (DTOs).

2.2 UML diagrams

2.2.1 Use case diagram

The interaction between the client player, the device sensors, and the Supabase backend is illustrated below.

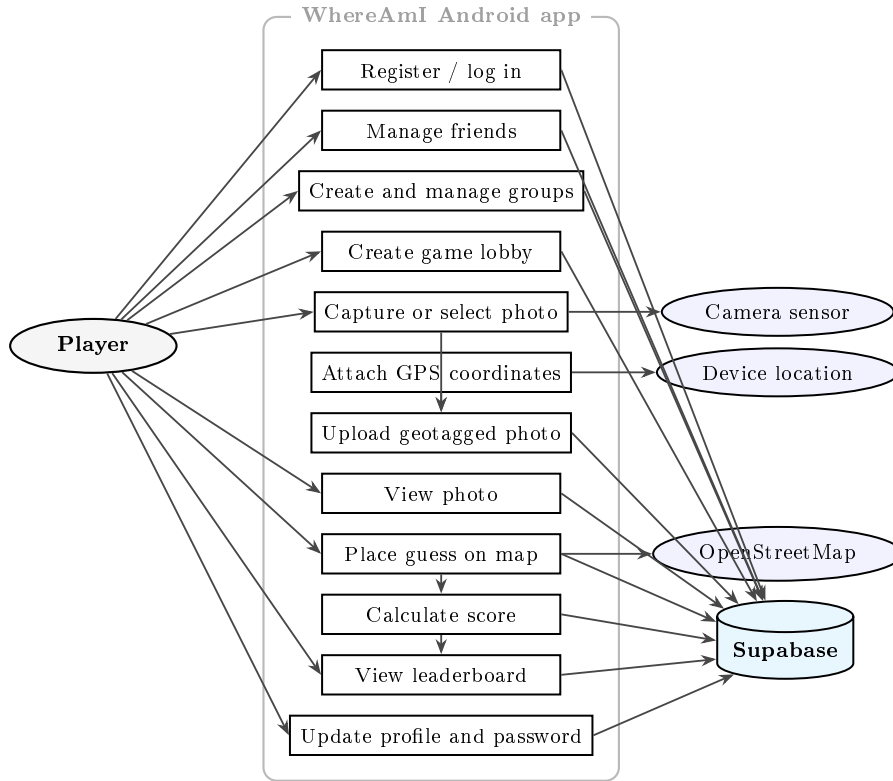


Figure 2: Use case diagram illustrating system actors and core actions.

2.2.2 Activity diagram

The following diagram details the sequence of activities during a single round of the game:

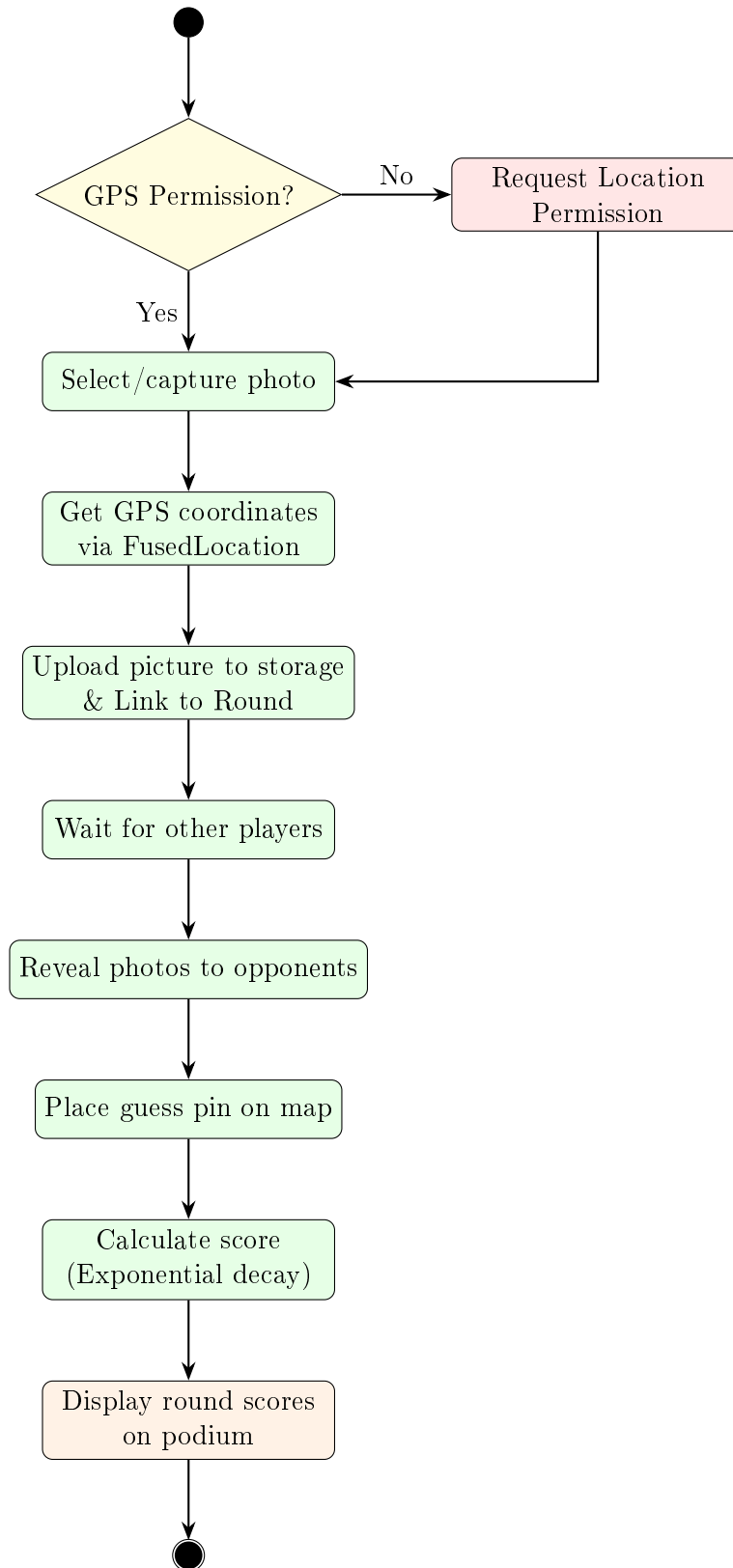


Figure 3: Activity diagram showing the lifecycle of a game round.

2.3 Database schema

The backend uses a relational database model implemented on PostgreSQL within Supabase. The database contains 10 tables designed to support user relationships, group structures, real-time messaging, game states, image meta-data, and score sheets.

The database layout is structured as follows:

1. **users:** Stores player profile information. When a user registers in Supabase Auth, a PostgreSQL trigger automatically copies the user ID, email, and metadata into this public table.
2. **groups & group_members:** Manages social groups and controls membership roles (Admin, Member).
3. **friendships:** Holds many-to-many user associations with states like **PENDING** or **ACCEPTED**.
4. **games:** Represents the overall match configuration, listing parameters like round counts, duration, and overall status (**CREATED**, **PLAYING**, **FINISHED**).
5. **rounds:** Tracks individual game slices. Each round is associated with a specific duration and a set of photo uploads.
6. **pictures:** Records picture metadata, storing the target's public image URL and the physical GPS coordinates (Latitude, Longitude) captured from the publisher's device.
7. **guesses:** Records a player's pinned coordinate guess, including the computed physical distance in meters and the final awarded points.
8. **game_scores & round_scores:** Aggregates points to display real-time leaderboards.

To ensure complete database integrity, structural diagrams were created during the conceptualization phase. The Conceptual Data Model (MCD) and Logical Data Model (MLD) diagrams are shown below.

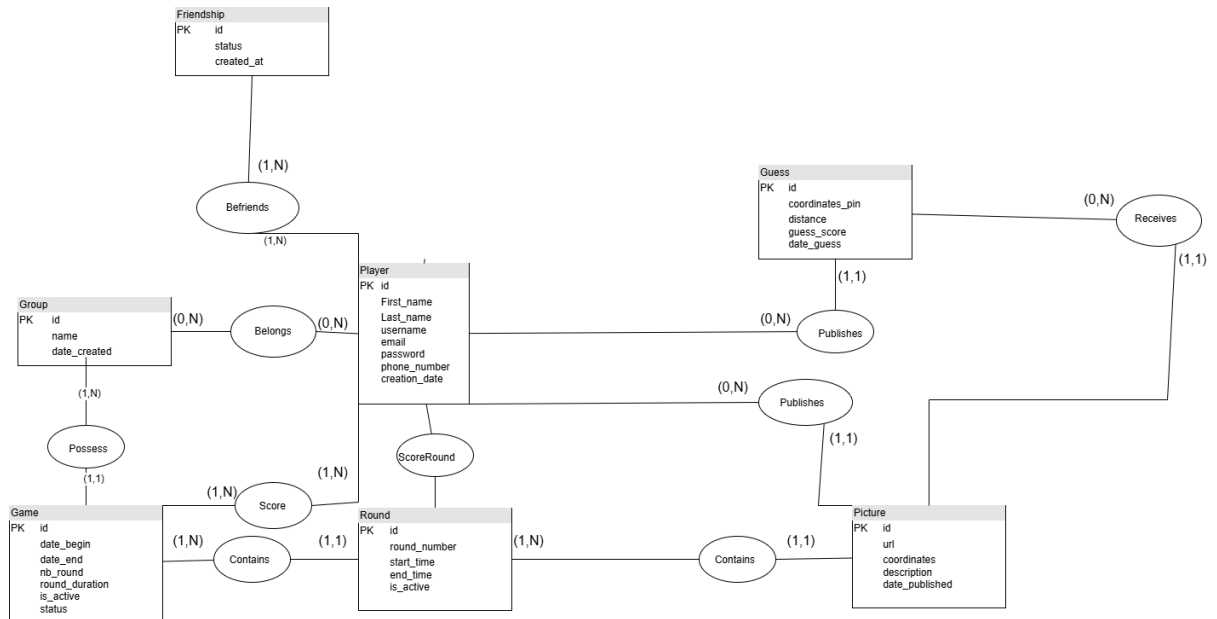


Figure 4: Conceptual Data Model (MCD) of the WhereAmI database.

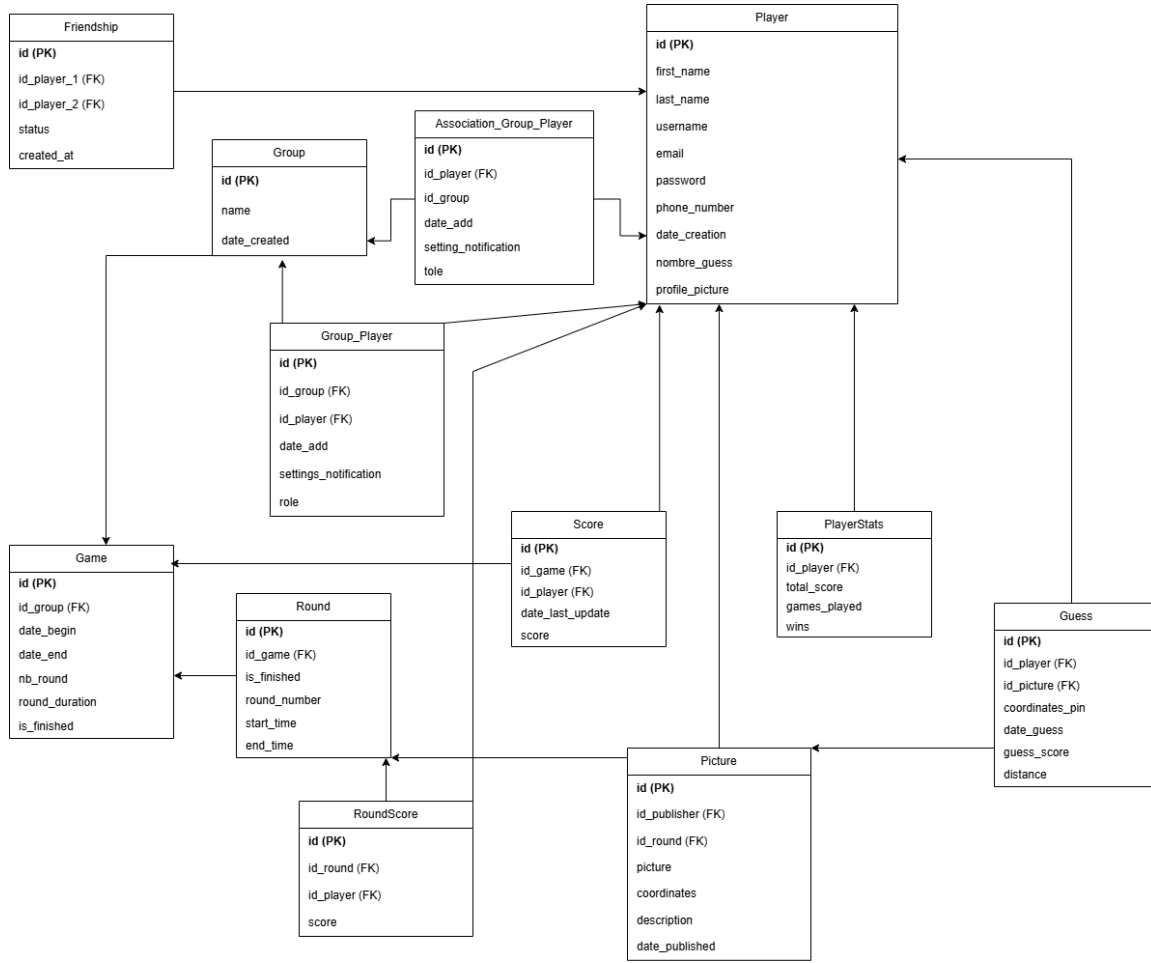


Figure 5: Logical Data Model (MLD) of the WhereAmI database.

3 Implementation details

3.1 Technologies and tools used

The implementation uses Android libraries and serverless technologies:

- **Kotlin & Jetpack Compose:** Used for core application logic and declarations of the Material Design 3 interface.
- **Supabase :** Provides the serverless framework, managing data storage (PostgreSQL), file storage (Supabase Storage for user pictures), and user sessions.
- **Ktor Client:** Used internally by Supabase for managing HTTP connections.
- **Google Play Services Location (21.3.0):** Integrates Android's location services to handle hardware sensors.
- **Osmdroid (6.1.18):** An open-source map engine used to display maps, add markers, capture touch coordinate inputs, and display guess feedback.
- **Coil Compose:** An asynchronous image loading library that fetches and caches game pictures from Supabase bucket URLs.

3.2 Key functionalities and implementation

3.2.1 Device location sensor integration

The GPS sensor is accessed through the Google Play Services Location library. Before uploading a photo, the app verifies location permissions. If granted, it requests the location via `FusedLocationProviderClient`:

```
1 val fusedLocationClient = LocationServices.getFusedLocationProviderClient(  
    context)  
2 fusedLocationClient.getCurrentLocation(  
3     Priority.PRIORITY_HIGH_ACCURACY,  
4     null  
5 ).addOnSuccessListener { loc: android.location.Location? ->  
6     if (loc != null) {  
7         viewModel.uploadPicture(LatLng(loc.latitude, loc.longitude), bytes)  
8     } else {  
9         viewModel.showError("Could not retrieve location. Please ensure GPS is  
10        turned on.")  
11    }  
12 }
```

Listing 1: Location sensing in RoundScreen.kt

3.2.2 Dynamic scoring calculation

The distance between the exact photo location and the player's guess is computed using the distance via `android.location.Location.distanceBetween`. The score decreases exponentially with distance:

$$S = \lfloor 5000 \cdot e^{-\frac{d}{100000}} \rfloor \quad (1)$$

where d is the distance in meters, and 5000 represents the maximum possible score (perfect guess).

```
1 val results = FloatArray(1)
2 Location.distanceBetween(
3     picture.location.latitude, picture.location.longitude,
4     guessLatitude, guessLongitude,
5     results
6 )
7 val distanceMeters = results[0].toDouble()
8 val score = (5000.0 * kotlin.math.exp(-distanceMeters / 100000.0)).toInt()
```

Listing 2: Scoring logic in SubmitGuessUseCase.kt

3.2.3 Interactive map and pinning

The Osmdroid map library is embedded in Compose using `AndroidView`. This allows users to place a pin by touching the map:

```
1 val mapEventsReceiver = object : MapEventsReceiver {
2     override fun singleTapConfirmedHelper(p: GeoPoint): Boolean {
3         if (!uiState.isSubmittingGuess) {
4             onPinLocationChanged(LatLng(p.latitude, p.longitude))
5         }
6         return true
7     }
8     override fun longPressHelper(p: GeoPoint): Boolean = false
9 }
10 val overlay = MapEventsOverlay(mapEventsReceiver)
11 mapView.overlays.add(overlay)
```

Listing 3: Map overlay interaction in RoundMapSubScreen.kt

3.2.4 Custom player avatars and animated podium

To enhance personalization custom profile pictures were implemented. Avatars are uploaded as raw byte arrays and stored in a Supabase Storage bucket, with public URL links saved in the PostgreSQL `users` profiles. A reusable `UserAvatar` component displays the profile picture or dynamically generates a circular canvas featuring the player's initials on a background color derived from their username.

These avatars are integrated directly into the Osmdroid interactive maps. When a round finishes, standard map markers are replaced with circular bitmap drawables showcasing each player's avatar over their guess coordinates. Furthermore, the scoreboard leaderboard displays a 3D-style podium with gold, silver, and bronze columns. The height of these columns grows dynamically from zero to their target layout heights upon entry, powered by Jetpack Compose's `animateDpAsState` with a staggered rank-based delay:

```
1 val animatedHeight by animateDpAsState(
2     targetValue = if (startAnimation) height.dp else 0.dp,
3     animationSpec = tween(
4         durationMillis = 1000,
5         delayMillis = (rank - 1) * 200 // Staggered: Gold (0ms), Silver (200ms),
6         Bronze (400ms)
7     )
8 )
```

```

6     ),
7     label = "podiumHeight"
8 )

```

Listing 4: Staggered podium animations in ScoresPodiumList.kt

3.2.5 Security via PostgreSQL Row Level Security (RLS)

Security policies restrict data visibility at the database level. For example, a player is only allowed to view pictures or guesses belonging to a game round they are actively participating in:

```

1 CREATE POLICY "Allow players to view pictures"
2 ON public.pictures FOR SELECT
3 TO authenticated
4 USING (
5     EXISTS (
6         SELECT 1 FROM public.rounds r
7         JOIN public.game_scores gs ON gs.game_id = r.game_id
8         WHERE r.id = round_id
9         AND gs.player_id = auth.uid()
10    )
11 );

```

Listing 5: RLS policies in schema.sql

3.3 Main challenges encountered

1. **Compose and legacy view interoperability:** Osmdroid MapView was built for legacy XML layout systems. Wrapping it in an `AndroidView` and managing its lifecycle (handling recompositions and state persistence of map pins) required custom handlers.
2. **Synchronizing game state:** As games progress, players upload images and submit guesses asynchronously. Standard database polling is inefficient and drains device batteries. The problem was solved by using Supabase Realtime Channels, which listen to table insert/update triggers and push updates to the client app.
3. **GPS timeout & accuracy:** Indoors or under weak network conditions, GPS sensors can return null values or take too long to obtain a lock. The app resolves this by specifying a high-accuracy priority parameter and handling null results with helpful system warnings instructing users to check their location settings.

4 Screenshots and user interface

4.1 Description of the main interfaces and navigation flow

The interfaces are structured to provide immediate feedback on game progress.

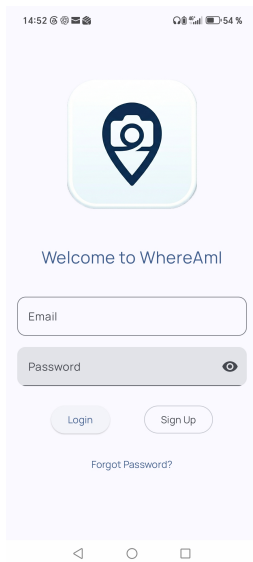
- **Login / password reset screens:** Clean text input fields with real-time field validation. The reset password screen utilizes deep link listeners ('whereami://reset-password') to automatically capture oauth recovery tokens.
- **Dashboard (HomeScreen):** Shows active game instances, a list of current rounds and whether the user needs to upload a picture or make guesses.
- **Friends and groups:** Tabbed navigation layouts showing active friends and groups. Users can invite members, create groups and games.
- **Round screen:** This screen manages the core gameplay loop and is split into four distinct sub-screens:
 1. **Player status sub-screen:** Displays a list of player cards indicating who has uploaded a picture and who has guessed.
 2. **Picture view sub-screen:** Shows the uploaded photo that players must guess. If the user hasn't uploaded their own photo, they cannot view others.
 3. **Map view sub-screen:** Embeds the OpenStreetMap where players place their guess pin and submit it. Once a round ends, this screen displays the exact location and all player guesses.
 4. **Scores sub-screen:** Shows the final round scoreboard.
- **Podium leaderboard (ScoresPodiumList):** A custom components screen displaying the top three players on colored pillars (Gold, Silver, Bronze) and list cards for players.

4.2 UI architecture flow

The application navigation architecture is managed via a centralized navigation graph in `AppNavGraph.kt`. Bottom Navigation items are visible on primary screens (Home, Groups, Friends, Account) and hidden during active games to maximize map space.

4.3 Application screenshots

The following screenshots illustrate the main user flows and interface states of the Android application.

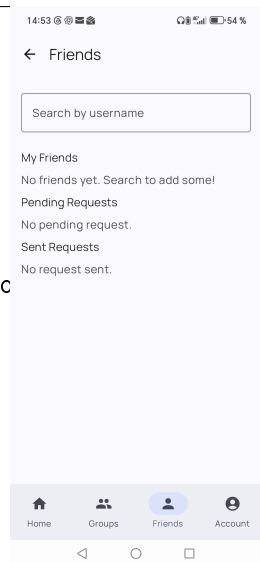


Login / registration

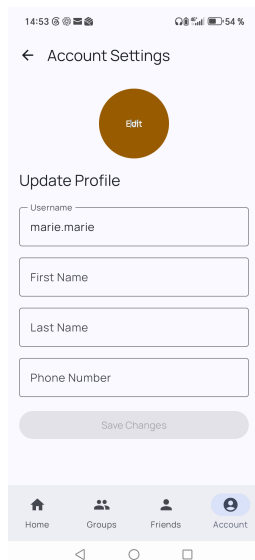


Home

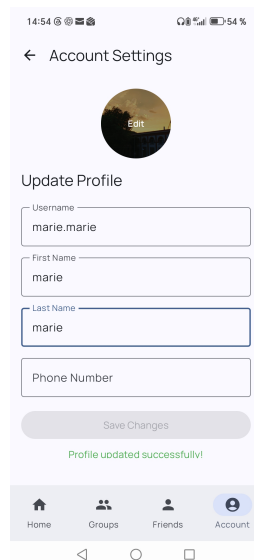
Groups



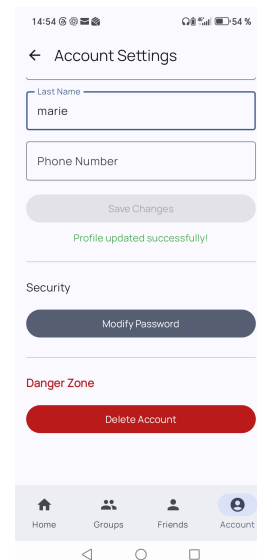
Group creation



Profile before update



Profile after update



Password update

Figure 6: Main application screens.

5 Testing and results

5.1 Test scenarios and validation

Quality assurance was achieved using instrumented unit tests that validate the business layer use cases. The complete multi-user game flow was validated in `GameLogicTest.kt` using mock database repositories to isolate logic from network latency.

Table 1: Core test scenarios covered in `GameLogicTest.kt`

Test phase	Input / Action	Expected outcome
1. Game creation	Create game in Group with 2 rounds	Game created, Round 1 state: CREATED
2. Media upload	Player A & B upload photos	Upload success, locations pinned
3. Guess submission	Player A guesses B's location (exact)	Score matches max: 5000 points
4. Math validation	Player B guesses A's location (10km off)	Score calculated: 4539 points (> 4000)
5. Round transition	Call <code>AdvanceRoundUseCase</code>	Round 1 marked FINISHED , Round 2 created
6. Leaderboard sync	Fetch game score sheets	Scores accumulated, rankings sorted

The main logic test is implemented as follows:

```
1 @Test
2 fun testWholeGameLogicFlow() = runBlocking {
3     // 1. Create Game
4     val settings = GameSettings(nbRound = 2, roundDurationMinutes = 120,
5     dateBegin = Clock.System.now(), dateEnd = Clock.System.now())
6     val createResult = createGameUseCase(groupId, settings)
7     assertTrue(createResult is CreateGameResult.GameCreated)
8
9     // 2. Upload pictures (Eiffel Tower and Louvre Museum coordinates)
10    val locationA = LatLng(48.8584, 2.2945)
11    val locationB = LatLng(48.8606, 2.3376)
12    gameRepo.uploadPicture(round0.id, playerA, locationA, ByteArray(0))
13    gameRepo.uploadPicture(round0.id, playerB, locationB, ByteArray(0))
14
15    // 3. Submit guesses (A guesses B exact -> 5000 pts. B guesses A slightly
16    off)
17    val guessAResult = submitGuessUseCase(round0.id, playerA, picB, picB.
18    location.latitude, picB.location.longitude)
19    assertEquals(5000, guessAResult.getOrNull().guessScore)
20
21    // 4. Advance Round & Validate Scores
22    val advanceResult = advanceRoundUseCase(game, round0)
23    val updatedGame = advanceResult.getOrNull()
24    assertEquals(1, updatedGame.currentRoundIndex)
25    assertEquals(5000, updatedGame.scoreSheets.find { it.playerId == playerA
26    }!!.score)
27 }
```

Listing 6: Complete game flow test (`GameLogicTest.kt`)

5.2 Bugs encountered and resolutions

- **Round timing accumulation:** Pre-allocated rounds in `CreateGameUseCase` accumulated leftover time if a round finished early.
Resolution: Shifted to dynamic round creation on the fly in `AdvanceRoundUseCase.kt` so that subsequent rounds start exactly when the previous one ends, calculating duration dynamically.
- **Zero scores on podium:** Guess info and `PlayerBox` models lacked score fields during UI state reconstruction, resetting scores to 0 when advancing rounds.

- Resolution:** Modified `AdvanceRoundUseCase.kt` to retrieve the original score-calculated guesses directly from the database using `getGuessesForRound()` before advancing.
- **Missing database RLS policies:** Scores did not appear on the podium because the `round_scores` table lacked read/write policies for authenticated users.
Resolution: Created SELECT, INSERT, and UPDATE Row-Level Security policies allowing authenticated players to manage round scores.
 - **Timer and round transition sync:** UI timers continued running after a round was completed, and users were not transitioned to the results.
Resolution: Monitored the round status in `RoundScreen.kt` to halt the countdown loop on `FINISHED`, auto-switched active tabs to the score view, and added a dynamic "Next Round" navigation button.
 - **Null location exception:** `FusedLocationProvider` returned null under weak connectivity, causing app crashes.
Resolution: Handled null states safely by checking GPS sensor availability first, displaying a toast message requesting user to toggle location services.
 - **PostgreSQL schema sync issue:** Auth registrations were not linked to the public `users` profiles.
Resolution: Created a PostgreSQL PL/pgSQL function trigger `handle_new_user()` that executes automatically upon inserting into the auth module.
 - **Osmdroid gesture conflict:** Touch actions on the Compose layout conflicted with Map scroll gestures.
Resolution: Configured the touch listener overlay in Osmdroid to consume tap gestures and update Compose states, while passing zoom events to the parent view.

6 Potential enhancements or future developments

While the application satisfies all project guidelines, several enhancements could expand its scope:

1. **Instant messaging within lobbies and groups:** Utilizing the database's `messages` and `reactions` tables to build a persistent chat panel directly inside the game round and group lobby screens, allowing players to exchange group messages in real-time.
2. **In-game push notifications:** Implementing background push notifications (such as notifying players when a round has been finished, when a new picture has been uploaded by a friend, or when the timer is close to expiring).
3. **Interactive guess modification:** Allowing players to update or fine-tune their active map pins multiple times up until the round timer officially expires.